

MANUAL DIDÁCTICO · UTN — FACULTAD REGIONAL MENDOZA

De Java a Python

POO por equivalencia

Guía de transferencia para quien ya modela y programa orientado a objetos en Java

El modelo no cambia · el mismo UML, dos implementaciones

Tabla maestra de equivalencias · qué se traduce, qué no, y por qué

Checklist de desintoxicación · los 7 java-ismos a erradicar

AUTOR

Mag. en Ing. de Software. **Alberto Cortez**

Análisis y Diseño Orientado a Objetos · Programación / Análisis de Sistemas

Índice

Del hábito Java al idioma Python, sin perder el diseño

Cómo usar este manual

La regla de oro del pasaje y para quién es esta guía

0 • El cambio de mentalidad

Tres inversiones que explican todo lo demás

1 • Tabla maestra de equivalencias

El mapa completo: concepto por concepto

2 • La property

Donde Python te libera de un dogma Java

3 • Encapsulamiento

De garantía del compilador a contrato de equipo

4 • Lo que se traduce mal

Streams, sobrecarga y los reflejos que fallan

5 • Trampas que Java no te enseñó a ver

Cuatro errores que en Java no pueden pasar

6 • Duck typing

Donde el diseño realmente cambia

7 • Regalos que Java no tiene

dataclass, herencia múltiple, operadores

8 • Checklist de desintoxicación

Los 7 java-ismos a buscar en tu código

Cierre • Qué es UML y qué es sintaxis

El experimento controlado de los dos manuales

Anexo A • Tabla de conversión rápida

Tipos, colecciones y operaciones

Anexo B • Autoevaluación y desafíos

Con soluciones

Anexo C • Glosario

Los términos del ecosistema Python, para quien viene de Java

Cómo usar este manual

Guía de transferencia · de Java a Python sin perder el diseño

Hay una manera fácil de aprender Python viniendo de Java, y es la que produce peores programadores. Consiste en abrir una tabla de equivalencias, memorizar que `extends` se escribe con paréntesis y que `System.out.println` se dice `print`, y empezar a escribir. El código compila. Los tests pasan. El code reviewer, si viene de Python, va a mirar el archivo con una mezcla de desconcierto y piedad, porque lo que tiene enfrente es Java disfrazado: getters en todas las clases, interfaces vacías declaradas para poder implementarlas, jerarquías de herencia cuyo único propósito era darle un tipo común al compilador — un compilador que, en Python, ni siquiera existe.

Este manual existe para evitar ese destino. Asume que ya sabés modelar y programar orientado a objetos en Java, y por eso no te enseña POO: te enseña a **reconocer lo que ya sabés** vestido con otra sintaxis, y —más importante— a identificar las tres o cuatro cosas donde el pasaje *no* es traducción sino rediseño. Esas tres o cuatro cosas son las que separan a alguien que escribe Python de alguien que escribe Java en Python.

La tesis del manual es la misma que sostiene la serie: **el modelo UML no cambia**. Los veintitrés diagramas del manual Java y del manual Python son idénticos, bit a bit. Lo que cambia es cómo cada lenguaje materializa las decisiones de diseño. Algunas cosas que en Java son sintaxis, en Python son convención. Otras que en Java son imposibles, en Python son gratis. Y unas pocas que en Java son obligatorias, en Python son opcionales — y ahí es donde hay que pensar de nuevo.

Para quién es este manual

Está escrito para tres lectores que se parecen más de lo que creen.

El **estudiante de la cátedra** que cursó Análisis y Diseño Orientado a Objetos con los ejercicios en Java y ahora encuentra los mismos once enunciados resueltos en Python. Su pregunta legítima es: si el diagrama es el mismo, ¿qué estoy aprendiendo de nuevo? La respuesta de este manual es que está aprendiendo a distinguir, por primera vez con evidencia, qué parte de lo que sabe era diseño y qué parte era ceremonia del lenguaje.

El **profesional con años de Java** que entra a un proyecto Python y descubre que su experiencia es simultáneamente su mayor activo y su principal obstáculo. Sabe modelar; ese conocimiento se transfiere íntegro. Pero también tiene reflejos automatizados durante años que en Python son ruido, y los reflejos no se desaprenden leyendo una tabla: se desaprenden entendiendo por qué existían.

El **docente** que tiene que explicar el pasaje sin caer en el «Python es más fácil» ni en el «Python es menos serio». Ninguna de las dos cosas es cierta, y ambas son cómodas.

Cómo recorrerlo

El manual tiene un orden pensado, aunque no todos los capítulos se leen igual.

El **capítulo 0** va antes que la **tabla maestra**, y esto no es una formalidad. Sin ese marco conceptual, la tabla del capítulo 1 parece una lista de sinónimos —y no lo es—. La tabla tiene una columna que dice «¿Equivalencia exacta?»,

y cada vez que esa columna dice **No**, hay un capítulo entero detrás explicando por qué. Leer la tabla sin el capítulo 0 es leer las respuestas sin las preguntas.

El **capítulo 1 es referencia, no lectura lineal**. Volvé a él cuando estés programando y necesites el equivalente de algo concreto. Nadie memoriza una tabla de cuarenta filas, y no hace falta.

Los **capítulos 2, 3 y 6 son los que cambian tu forma de diseñar**. El resto del manual es traducción; esos tres son conversión. Si tenés poco tiempo, son esos. El 2 te saca el hábito más arraigado (los getters), el 3 te reubica el encapsulamiento en un lugar distinto de la arquitectura, y el 6 te obliga a justificar cada herencia por el dominio en lugar de por el compilador.

El **capítulo 5 es el más urgente en la práctica**. Son cuatro errores que Java te *impedía* cometer, así que tu intuición de programador Java no los detecta. No los vas a ver venir, y dos de ellos producen bugs que se manifiestan lejos del lugar donde los escribiste. Es el capítulo que conviene leer antes de escribir la primera clase, no después del primer bug.

El **capítulo 8 es un checklist de code review**. Tuya y de tus estudiantes. Está diseñado para usarse con el archivo abierto al lado, no para leerse una vez.



Figura 1 · Recorrido del manual: seis capítulos de traducción y tres de conversión (2, 3 y 6), donde el diseño se piensa de nuevo.

LA REGLA DE ORO DEL PASAJE

En Java diseñás **contra el compilador**; en Python diseñás **contra el lector**. Todo lo que en Java te obligaba la sintaxis, en Python lo sostenés vos con disciplina. El diseño orientado a objetos no se relaja: se vuelve responsabilidad tuya en lugar de responsabilidad del javac.

Esta frase se puede leer como una advertencia o como una promesa, y conviene leerla como las dos cosas a la vez. Como advertencia, dice que Python te va a dejar hacer barbaridades que Java te prohibía. Como promesa, dice que Python te va a dejar expresar diseños que Java te impedía expresar. Ambas son verdad, y la diferencia entre un buen programador Python y uno malo es exactamente cuál de las dos capacidades ejerce.

0 El cambio de mentalidad

Tres inversiones que explican casi todo lo demás

Antes de cualquier tabla de equivalencias hay que entender tres inversiones conceptuales. Si las internalizás, la mayoría de las diferencias sintácticas se deducen solas y la tabla del capítulo 1 se vuelve casi innecesaria. Si no lo hacés, vas a escribir Java con sintaxis de Python durante años, y lo peor es que nadie te lo va a decir, porque el código funciona.

En Java	En Python
El compilador impone. El tipo es una barrera estática. <code>private</code> es una garantía.	El programador acuerda. El tipo es documentación. <code>_privado</code> es un pacto entre adultos.
Herencia para reusar tipo. Necesitás una superclase común para tratar objetos igual.	Duck typing. Si tiene el método, sirve. La superclase común es opcional.
La clase declara todo por adelantado.	El objeto se arma en runtime.

Primera inversión · de la imposición al acuerdo

En Java, cuando escribís `private int saldo`, estás haciendo una afirmación que el compilador va a hacer cumplir por vos. No es una sugerencia ni una nota al lector: es una barrera. Si alguien en otro paquete intenta tocar ese campo, el código no compila. Punto. El diseñador Java delega el cumplimiento de sus decisiones en una herramienta que no negocia.

En Python, cuando escribís `self._saldo`, estás haciendo exactamente la misma afirmación de diseño —«esto es interno, no lo toques»— pero no hay nadie que la haga cumplir. El guión bajo es un cartel. Un cartel que la comunidad entera respeta, que los linters marcan, que los code reviewers señalan, y que el intérprete ignora por completo. Si alguien escribe `cuenta._saldo = 999`, funciona.

El reflejo del programador Java al descubrir esto es concluir que Python «no tiene encapsulamiento». Es una conclusión comprensible y equivocada. El encapsulamiento es una *decisión de diseño*, no una característica del lenguaje. Java te da una herramienta para hacerla cumplir automáticamente; Python te pide que la hagas cumplir con convención, herramientas externas y cultura de equipo. La decisión —qué es interno y qué es público— la tomás vos en los dos casos, y es exactamente la misma decisión.

Lo que cambia es dónde vive el rigor. En Java vive en el lenguaje. En Python vive en el equipo. Ninguno de los dos modelos es intrínsecamente más disciplinado: hay bases de código Java con `public` en todos lados y bases de código Python con encapsulamiento impecable, y viceversa. Pero el modelo de Python es más honesto respecto de una verdad incómoda: el encapsulamiento de Java tampoco es absoluto. Existe la reflexión, existe `setAccessible(true)`, y cualquier framework de serialización la usa todos los días. La diferencia es de grado, no de naturaleza.

Segunda inversión · de la herencia al duck typing

Esta es la que tiene consecuencias más profundas en el diseño, y la que se explica entera en el capítulo 6. Acá basta con plantear el punto.

En Java, si querés recorrer una lista de objetos y llamarles el mismo método, **necesitás** que tengan un tipo común. No es una recomendación de diseño: es un requisito del compilador. Sin una superclase o una interfaz compartida, no hay `List<Algo>` que los contenga, y sin esa lista no hay polimorfismo. Por eso, en Java, muchas jerarquías de herencia existen no porque el dominio diga que esas cosas son parientes, sino porque el compilador exige un tipo común para poder tratarlas igual.

En Python eso no hace falta. Si el objeto tiene el método, se le puede llamar. La lista puede contener cualquier cosa. El polimorfismo es gratis y no pide permiso.

Acá aparece la trampa, y es sutil: al descubrir esto, el programador Java tiende a concluir que en Python la herencia sobra. No sobra. Lo que sobra es la herencia *que existía solo para satisfacer al compilador*. La herencia que existe porque el dominio afirma que un triángulo es un polígono sigue estando plenamente justificada, y borrarla es empobrecer el modelo. La pregunta que hay que aprender a hacerse es: ¿esta superclase está acá porque el negocio dice que estas cosas son un mismo concepto, o está acá porque sin ella no compilaba? En Java las dos razones se confunden porque producen el mismo código. En Python se separan, y esa separación es la que revela cuál era cuál.

Tercera inversión · de la declaración al runtime

En Java, una clase es un contrato cerrado en tiempo de compilación. Los campos que declaraste son los campos que hay. Los métodos que escribiste son los métodos que existen. Un objeto de esa clase no puede, en ejecución, adquirir un atributo que no estaba previsto.

En Python, la clase es un punto de partida y el objeto se termina de armar en ejecución. Podés agregarle atributos a una instancia después de creada. Podés reemplazar un método. Podés preguntarle a un objeto qué métodos tiene y decidir en runtime. Esta flexibilidad es la base de gran parte del ecosistema —los ORMs, los frameworks de test, la serialización— y es también la razón por la que un `typo` en un nombre de atributo no produce un error de compilación sino un objeto silenciosamente distinto del que esperabas.

El corolario práctico es que en Python muchas cosas que Java resuelve en tiempo de compilación se resuelven con herramientas: `mypy` para los tipos, los linters para las convenciones, los tests para el resto. La red existe, pero no viene puesta: hay que instalarla.

Por qué esto importa antes que la sintaxis

La inversión de fondo, la que unifica a las tres, es esta: **Java pone el conocimiento del diseño en el lenguaje** —el modificador, la firma, la anotación, la declaración— mientras que **Python lo pone en el equipo**: la convención, el linter, el type checker, el code review, el manual de la cátedra.

Ninguno de los dos es más riguroso que el otro. Cambia dónde vive el rigor. Y como cambia de lugar, un programador que trae el hábito de un lenguaje al otro sin darse cuenta de la mudanza termina buscando el rigor donde ya no está —y no encontrándolo— o dejando de ejercerlo porque el compilador ya no se lo reclama.

Dónde vive el rigor

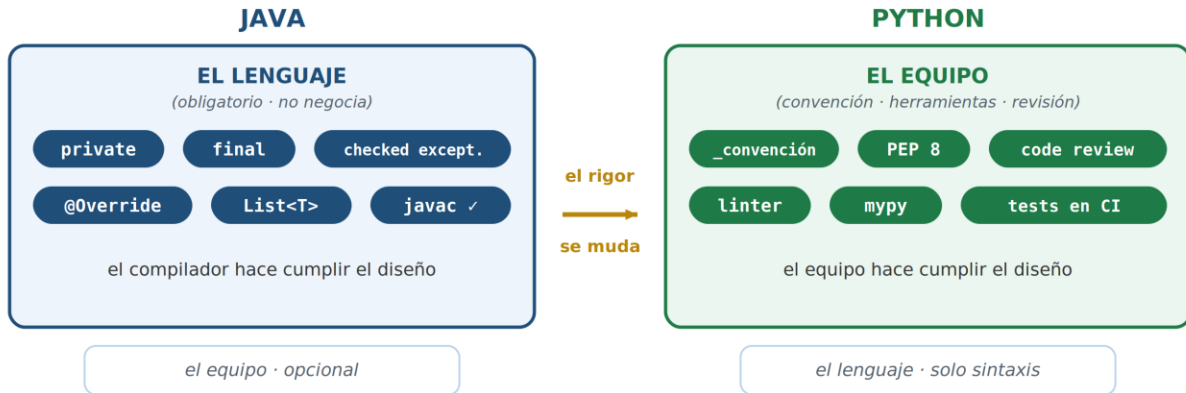


Figura 2 · La mudanza del rigor: en Java lo hace cumplir el lenguaje; en Python, el equipo y sus herramientas. La decisión de diseño es la misma.

Con esto en la cabeza, la tabla del próximo capítulo se lee distinto. No es una lista de traducciones: es un mapa de dónde el diseño se conserva idéntico y dónde hay que volver a pensarlo.

1 Tabla maestra de equivalencias

El mapa completo · concepto por concepto

Este capítulo es referencia. No está pensado para leerse de corrido sino para consultarse mientras programás, y por eso está partido en cuatro tablas temáticas en lugar de una sola de cuarenta filas.

La columna decisiva es la última. Cuando dice **Sí**, estás traduciendo: cambia la sintaxis y nada más, podés mover el conocimiento de un lenguaje al otro sin pensar. Cuando dice **No**, estás cambiando de modelo mental, y ahí es donde se cometen los errores — porque la sintaxis nueva funciona pero significa algo distinto de lo que creés. Cada **No** de estas tablas tiene un capítulo detrás que lo explica.

El mapa de la tabla maestra

la columna «¿Exacta?» divide el pasaje en tres zonas



Figura 3 · Las tres zonas del pasaje. La verde se cruza sin pensar; la roja es donde un hábito Java produce un error Python.

Estructura de clase

Concepto OO	Java	Python	¿Exacta?
Clase	<code>public class Figura { }</code>	<code>class Figura:</code>	Sí
Constructor	<code>public Figura(String n) { }</code>	<code>def __init__(self, n: str) -> None:</code>	Sí. Pero <code>__init__</code> inicializa, no crea (eso es <code>__new__</code>)
this	Implicito	<code>self</code> — explícito, siempre 1er parámetro	Conceptual sí, sintáctico no
Herencia	<code>class Triangulo extends Poligono</code>	<code>class Triangulo(Poligono):</code>	Sí
<code>super()</code>	<code>super(n);</code>	<code>super().__init__(n)</code>	Sí — pero NO es automático
Clase abstracta	<code>public abstract class Poligono</code>	<code>class Poligono(ABC):</code>	Sí
Método abstracto	<code>protected abstract int ladosEsperados();</code>	<code>@abstractmethod</code>	Sí

Concepto OO	Java	Python	¿Exacta?
Interfaz	interface Comparable { }	Protocol / ABC pura	No exacto — ver cap. 6
@Override	Anotación verificada	(nada)	No existe. Typo silencioso

Dos filas de esta tabla merecen comentario porque son fuente habitual de bugs.

El `super()` no es automático. En Java, si tu constructor no llama explícitamente a `super(...)`, el compilador inserta una llamada al constructor sin argumentos del padre. Es invisible pero está. En Python no hay nada de eso: si no escribís `super().__init__()`, el constructor del padre simplemente no corre. El objeto queda a medio construir, sin los atributos que el padre iba a inicializar, y el error aparece mucho después —en otro método, en otro archivo— cuando algo intenta leer un atributo que nunca existió. Es de los bugs más desorientadores del pasaje.

El `@Override` no existe. En Java, la anotación le pide al compilador que verifique que efectivamente estás sobrescribiendo algo. Si te equivocás en el nombre o en la firma, no compila. En Python no hay equivalente: si querés redefinir `calcular_area` y escribís `calcular_aera`, acabás de crear un método nuevo. El padre sigue respondiendo con su implementación, tu método nunca se llama, y no hay ningún error en ninguna parte. Python 3.12 agregó `@typing.override`, que mypy verifica — pero es opcional y hay que acordarse de ponerlo.

PYTHON — EL TYPO SILENCIOSO Y SU RED

```
class Poligono:
    def calcular_area(self) -> float: ...

class Triangulo(Poligono):
    def calcular_aera(self) -> float: # typo: creó un método NUEVO
        return self.base * self.altura / 2

# Triangulo().calcular_area() llama al del padre. Sin error, nunca.

# Desde Python 3.12, la red equivalente al @Override:
from typing import override

class Triangulo(Poligono):
    @override
    def calcular_aera(self) -> float: # ahora mypy Sí lo marca
        ...
```

Visibilidad y estado

Concepto OO	Java	Python	¿Exacta?
public	public int x;	<code>self.x</code>	Sí (default)
protected	protected int x;	<code>self._x</code> — convención	No. Nada lo impide
private	private int x;	<code>self.__x</code> — name mangling	No. Solo ofusca
final (campo)	private final List<Lado> lados	<code>Final[list[Lado]]</code> o property sin setter	No. Anotación, no barrera

Concepto OO	Java	Python	¿Exacta?
static	<code>public static int contar()</code>	<code>@staticmethod</code>	Sí
Campo de clase	<code>private static int total;</code>	<code>total: int = 0</code> (nivel clase)	Sí — ojo mutables (cap. 5)
Getter / Setter	<code>getNroLados()</code> / <code>setX()</code>	<code>@property</code> / <code>@x.setter</code>	Mejor en Python — cap. 2

Esta tabla es la más roja del manual, y no es casualidad: la visibilidad es donde Java y Python discrepan más. Tres de las siete filas dicen que no hay equivalencia real. El capítulo 3 se ocupa entero de esto, pero conviene registrar desde ahora que **ninguna construcción de Python reproduce el private de Java** — ni el guión bajo simple, ni el doble, ni ninguna combinación. Lo más parecido que existe es una property de solo lectura que devuelve una copia, y ni siquiera eso impide el acceso al atributo interno.

Comportamiento y tipos

Concepto OO	Java	Python	¿Exacta?
Sobrecarga	<code>area(int)</code> , <code>area(int,int)</code>	Default args, <code>*args</code> , <code>@singledispatch</code>	No existe
Genéricos	<code>List<Lado></code>	<code>list[Lado]</code> — hint, no verificado	No. Documentación + mypy
<code>toString()</code>	<code>@Override public String toString()</code>	<code>__repr__</code> / <code>__str__</code>	Sí
<code>equals()/hashCode()</code>	Par obligatorio	<code>__eq__</code> / <code>__hash__</code>	Sí — misma trampa del par
Comparable	<code>compareTo()</code>	<code>__lt__</code> + <code>@total_ordering</code>	Sí
Excepción	<code>throw new IllegalArgumentException(m)</code>	<code>raise ValueError(m)</code>	Sí
Checked exceptions	<code>throws IOException</code>	(no existe)	No. Ninguna es checked
Herencia múltiple	Solo de interfaces	De clases, permitida (MRO/C3)	Python puede más

La sobrecarga no existe, y esto sorprende. Si definís dos métodos con el mismo nombre y distinta firma, el segundo pisa al primero. Sin error, sin advertencia. Python resuelve las llamadas por nombre, no por firma, así que el concepto de «misma operación, distintos parámetros» se expresa de otra manera: con argumentos por defecto cuando las variantes son pocas y compatibles, con `*args` cuando la cantidad es variable, con `@classmethod` cuando lo que querés son constructores alternativos —el caso más frecuente— y con `@singledispatch` cuando el comportamiento realmente depende del tipo del argumento. El capítulo 4 muestra el patrón.

La trampa del par `equals()/hashCode` se conserva idéntica. En Java, si sobrescribís `equals` sin sobrescribir `hashCode`, tu objeto se comporta mal en cualquier `HashMap` o `HashSet`. En Python la trampa es la misma pero con un agravante: si definís `__eq__` y no definís `__hash__`, Python *anula* el hash — tu clase pasa a ser no hasheable y revienta al meterla en un set. Es la misma regla de diseño de siempre, con un castigo más ruidoso.

Colecciones y organización

Concepto OO	Java	Python	¿Exacta?
ArrayList<T>	<code>new ArrayList<>()</code>	<code>[]</code>	Sí
Map<K,V>	<code>new HashMap<>()</code>	<code>{}</code>	Sí
Streams	<code>.stream().filter().map()</code>	Comprehensions	Conceptual sí — cap. 4
Paquete	<code>package ar.utn.figuras;</code>	Módulo = archivo <code>.py</code>	Aproximado

La última fila dice «aproximado» por una razón que conviene tener presente al organizar un proyecto: en Java el paquete es una jerarquía de directorios que se corresponde con el nombre, y una clase pública vive en su propio archivo. En Python el módulo es el archivo, y un archivo puede contener tantas clases como tenga sentido agrupar. Un módulo `figuras.py` con `Figura`, `Poligono`, `Triangulo` y `Lado` no es desprolijidad: es la organización idiomática. Traer la regla de «una clase, un archivo» desde Java produce proyectos Python fragmentados en decenas de archivos de quince líneas, y es uno de los java-ismos más visibles a simple vista.

2 La property

Donde Python te libera de un dogma Java

Este es **el cambio de hábito más importante** de todo el manual. Si te llevás una sola cosa, que sea esta.

Por qué Java te enseñó a escribir getters

Vale la pena empezar reconociendo que el hábito del getter, en Java, **es correcto**. No es superstición ni burocracia: es defensa propia, y responde a un problema real.

Supongamos que exponés `public int nroLados` y medio sistema escribe `figura.nroLados`. Un día descubriste que ese número tiene que calcularse, o validarse, o registrarse en un log. Necesitás lógica. Y en Java, meter lógica implica convertirlo en método, lo que implica que todos los clientes pasen a escribir `figura.getNroLados()`. Cambió la sintaxis de acceso: todo lo que dependía de esa clase deja de compilar. Si la clase es parte de una API pública, acabás de romper a todos tus usuarios.

El getter preventivo resuelve eso. Escribís `getNroLados()` desde el día uno, aunque hoy solo haga `return lados.size()`, porque así el día que necesite lógica la agregás adentro y nadie se entera. El getter es **un seguro contra el futuro**. El precio es escribir —y leer— miles de métodos que no hacen nada. La comunidad Java aceptó ese precio, con razón, porque la alternativa era peor.

Python no tiene ese problema, y por lo tanto no necesita ese seguro.

La property: cambiar la implementación sin cambiar la interfaz

En Python podés convertir un atributo en método sin que ningún cliente lo note. Empezás con `self.nro_lados = 3`; el día que necesita lógica, lo convertís en `@property`. El código que hacía `figura.nro_lados` sigue funcionando exactamente igual, sin recompilar, sin tocar una línea. La sintaxis de acceso es la misma para un atributo y para una property: **el cliente nunca supo la diferencia y nunca la va a necesitar saber**.

Esto da vuelta la regla. El getter preventivo dejó de comprar nada, porque el riesgo del que te protegía ya no existe. Y lo que antes era prudencia ahora es ruido.

Cambiar la implementación sin romper al cliente

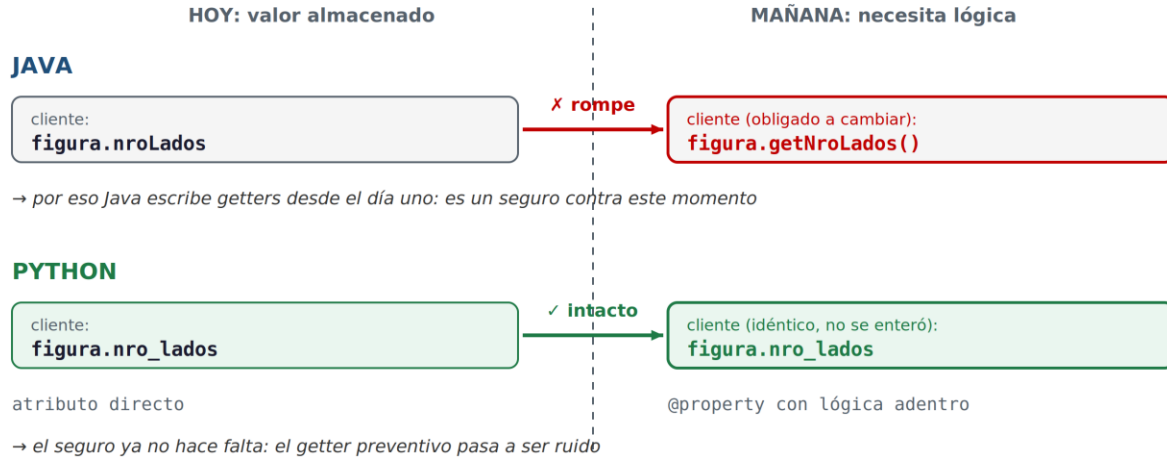


Figura 4 · El momento contra el que el getter Java asegura. En Python ese momento no rompe nada: el cliente escribe lo mismo antes y después.

El mismo modelo, dos implementaciones

Tomado del Ejercicio 1 de la serie — **Poligono**, con el atributo derivado `/nroLados` y la restricción refinada por subclase. Es el mismo diagrama en los dos manuales; lo único que cambia es el código:

JAVA

```
public abstract class Poligono extends Figura {
    private final List<Lado> lados = new ArrayList<>();

    public void agregarLado(Lado lado) { lados.add(lado); }

    /** Atributo derivado: se calcula, no se almacena. */
    public int nroLados() { return lados.size(); }

    /** Cada subclase concreta refina la cantidad exacta. */
    protected abstract int ladosEsperados();

    public boolean estructuraValida() {
        int esperados = ladosEsperados();
        if (esperados == -1) return nroLados() >= 3;
        return nroLados() == esperados;
    }
}
```

PYTHON

```
from abc import ABC, abstractmethod

class Poligono(Figura, ABC):
    def __init__(self) -> None:
        super().__init__()
        self._lados: list[Lado] = []

    def agregar_lado(self, lado: Lado) -> None:
        self._lados.append(lado)

    @property
    def nro_lados(self) -> int:      # derivado: se calcula
        return len(self._lados)

    @abstractmethod
    def lados_esperados(self) -> int:
        """Cada subclase concreta refina la cantidad exacta."""

    @property
    def estructura_valida(self) -> bool:
        esperados = self.lados_esperados()
        if esperados == -1:
            return self.nro_lados >= 3
        return self.nro_lados == esperados
```

Mirá qué se conservó y qué se movió. La **jerarquía** es la misma. El **método abstracto** que cada subclase refina es el mismo. La **lógica de validación** es idéntica, línea por línea. Lo único que cambió de naturaleza es `nroLados()`, que era un método y pasó a ser algo que se *lee* como atributo.

Una aclaración sobre el encabezado, porque genera una pregunta legítima en clase: `class Poligono(Figura, ABC)` lleva `ABC` para que el ejemplo funcione aislado. Si `Figura` ya hereda de `ABC` —como ocurre en el manual de la serie— alcanza con `class Poligono(Figura):`, porque la condición para que `@abstractmethod` tenga efecto es que haya un `ABC` en algún punto de la cadena de herencia, no en el encabezado inmediato. Las dos formas son correctas; lo que no funciona es `@abstractmethod` en una clase sin ningún `ABC` en su ascendencia: ahí el decorador se ignora en silencio y la clase se deja instanciar incompleta.

El punto arquitectónico

`figura.nro_lados` se lee como acceso a campo pero ejecuta un método. Y acá aparece algo que vale más que una comodidad sintáctica: en el diagrama UML, `/nroLados` está marcado como **atributo derivado**. La barra inclinada significa exactamente eso: es un atributo —se lee como atributo, se usa como atributo— cuyo valor se calcula en lugar de almacenarse.

En Java, ese atributo derivado del modelo se convierte necesariamente en un método. Pierde su cara de atributo en la traducción. El lector del código Java ve `figura.nroLados()` y tiene que reconstruir mentalmente que eso, en el modelo, era un `/atributo`. En Python conserva la cara: `figura.nro_lados`.

La sintaxis Python está más cerca del UML que la Java. El / del diagrama se corresponde con `@property`. Esto no es un detalle estético: significa que el código Python conserva una distinción que el modelo hacía y que el código Java pierde. Cuando enseñás atributos derivados con el manual Java, tenés que explicar que el / «se implementa como método». Con el manual Python, el / se implementa como lo que es.

La regla, invertida

En Java	En Python
Escribí getters siempre, por las dudas.	Exponé el atributo directo. Ponés <code>@property</code> el día que necesitás lógica.

Escribir getters preventivos en Python es un java-ismo, y el code reviewer te lo va a marcar. No es cuestión de estilo. El argumento de fondo es que **el ruido oculta la señal**: si todos los accesos pasan por un `get_x()` idéntico, el lector no tiene forma de saber cuáles tienen lógica detrás y cuáles no, y termina no leyendo ninguno. Cuando solo los que hacen algo son properties, el `@property` se vuelve informativo — dice «acá pasa algo, prestá atención». Un decorador que está en todos lados no informa nada.

Cuándo sí escribir una property

La regla «no escribas getters» no significa «nunca uses properties». La property se justifica cuando hay algo que decir:

- **Atributo derivado.** El caso de `nro_lados`: el valor se calcula a partir de otros. Es el uso canónico y el que aparece en el modelo UML con la barra.
- **Validación al asignar.** Un `@x.setter` que rechaza valores inválidos. Es donde va la restricción del enunciado que no se dibuja como caja.
- **Solo lectura.** Una property sin setter es el equivalente más cercano al campo `final` expuesto: el cliente lee, no escribe.
- **Cálculo costoso que conviene cachear.** Con `@cached_property`, el valor se computa una vez y queda. En Java esto es un campo más `lazy init` a mano.

Si ninguna de esas cuatro aplica, el atributo va directo y punto. Y el día que aplique alguna, la property entra sin romper a nadie — que es, después de todo, el argumento entero de este capítulo.

Así se ve «el día que necesitás lógica». Supongamos que `Lado` empezó exponiendo `self.longitud` directo, y el enunciado agrega la restricción de que un lado no puede medir cero o menos — una de esas reglas que en el diagrama van como nota entre llaves, no como caja:

PYTHON

```
class Lado:
    def __init__(self, longitud: float) -> None:
        self.longitud = longitud    # pasa por el setter de abajo

    @property
    def longitud(self) -> float:
        return self._longitud

    @longitud.setter
    def longitud(self, valor: float) -> None:
        if valor <= 0:
            raise ValueError("un lado debe medir más que cero")
        self._longitud = valor

# El cliente escribe lo mismo que siempre:
lado.longitud = 5.0    # ✓ valida y asigna
lado.longitud = -1     # X ValueError – la regla del enunciado
```

Fijate dos cosas. Primero, el cliente que hacía `lado.longitud = 5.0` antes de que existiera la validación escribe exactamente lo mismo después: la restricción entró sin romper a nadie. Segundo, hasta el propio `__init__` asigna a través del setter, así que la regla se aplica también al construir — no hay puerta trasera. Este es el destino natural de las restricciones del enunciado que el manual de la serie enseña a no dibujar como cajas: terminan viviendo en un setter o en un `raise` del constructor.

3 Encapsulamiento

De garantía del compilador a contrato de equipo

Esta es la mesa donde más incomodidad vas a sentir, y conviene atravesarla de frente en vez de buscarle un atajo. Python tiene tres niveles de visibilidad y **ninguno** es lo que Java llama `private`. No hay un equivalente escondido ni un truco que lo reproduzca. Simplemente no está.

PYTHON

```
class Cuenta:
    def __init__(self) -> None:
        self.saldo = 0          # público
        self._interno = 0      # "protected" – convención
        self.__mangled = 0     # name mangling
```

- `_interno`: **nada** te impide tocarlo desde afuera. Es un cartel de «no entres». Los linters lo marcan, los IDEs lo esconden del autocompletado, la comunidad lo respeta; el intérprete lo ignora.
- `__mangled`: se renombra a `_Cuenta__mangled`. **No es privado** — es antichoque de nombres en herencia, un mecanismo pensado para que una subclase no pise sin querer un atributo interno del padre.
c. `_Cuenta__mangled` funciona perfecto desde cualquier lado.

El error frecuente es usar el doble guión creyendo que es el `private` de Java. No lo es, y además tiene un costo: rompe la herencia legítima —una subclase no puede acceder al atributo del padre sin escribir el nombre mangleado— y complica el debugging y los tests. La convención de la comunidad es clara: **usá un guión bajo simple**. El doble se reserva para el caso puntual de evitar colisiones en jerarquías profundas, que es para lo que fue diseñado.

Vale la pena ver ese caso puntual una vez, para que el mecanismo deje de ser misterioso. El mangling existe para que padre e hijo puedan usar el mismo nombre interno sin pisarse:

PYTHON — PARA QUÉ EXISTE EL DOBLE GUIÓN

```
class Figura:
    def __init__(self) -> None:
        self.__cache = None          # se guarda como _Figura__cache

class Poligono(Figura):
    def __init__(self) -> None:
        super().__init__()
        self.__cache = {}           # se guarda como _Poligono__cache

# Sin mangling, la asignación del hijo pisaría la del padre.
# Con mangling, conviven: son dos atributos distintos.
```

Eso es todo lo que el doble guión hace: renombrar por clase para que no haya choque. Leído así queda claro por qué usarlo «para privacidad» es un error de concepto — el renombre es predecible y público, no una barrera.

Estructural vs. cultural

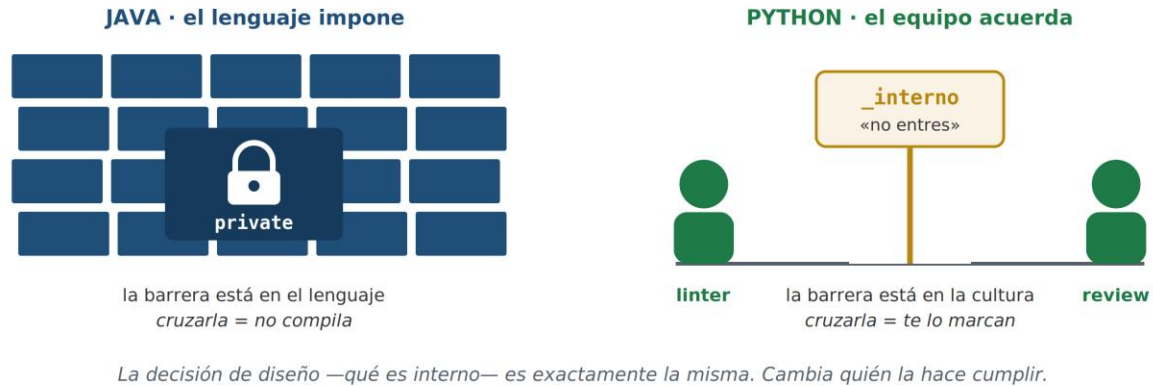


Figura 5 · La misma frontera, dos mecanismos: en Java es un muro del lenguaje; en Python, un cartel que el equipo y sus herramientas hacen respetar.

El caso del final

El `private final List<Lado> lados` del Ejercicio 1 dice dos cosas a la vez, y conviene separarlas porque Python las trata distinto. Dice «nadie de afuera toca esta lista» (`private`) y dice «esta referencia no se reasigna nunca» (`final`). Notá que el `final` de Java protege la *referencia*, no el *contenido*: `lados.add(x)` compila perfectamente en un campo `final`. Es una garantía más débil de lo que parece.

Python ofrece tres aproximaciones, de menos a más fuerte:

PYTHON

```
# 1. Documentar la intención (mypy lo verifica, el runtime no)
from typing import Final
self._lados: Final[list[Lado]] = []

# 2. Property de solo lectura → protege la referencia, no el contenido
@property
def lados(self) -> list[Lado]:
    return list(self._lados)    # copia defensiva: el cliente no muta

# 3. Inmutabilidad real
from dataclasses import dataclass

@dataclass(frozen=True)
class Lado:
    longitud: float
```

La primera es pura documentación: `Final` le dice a `mypy` que no querés reasignar, y `mypy` te avisa si lo hacés. En ejecución no pasa nada. Es el equivalente honesto del `final` de Java en cuanto a fuerza real de la garantía, y de hecho es más útil, porque `mypy` te avisa en el editor.

La segunda es la más importante en la práctica y la que más conviene incorporar: **property de solo lectura que devuelve una copia**. Es más fuerte que el `final` de Java, porque el `final` dejaba que el cliente hiciera `getLados().add(x)` si el getter devolvía la lista interna. Acá el cliente recibe una copia: puede mutarla todo lo que quiera y el objeto no se entera. La lista interna sigue intacta.

La tercera es la única inmutabilidad real, y es más fuerte que cualquier cosa que Java te dé sin escribir código: `frozen=True` hace que asignar a un campo tire `FrozenInstanceError`. Sirve para los objetos-valor del modelo — Lado, Punto, Medida—, esos que se comparan por contenido y no tienen identidad propia. Ese criterio, el de *value object* contra *entity*, es el mismo que aplicabas en Java.

TRADUCCIÓN PARA UN ARQUITECTO

En Java el encapsulamiento es **estructural** (lo impone el lenguaje). En Python es **cultural** (lo impone el equipo, el linter y el code review). El diseño no cambia: cambia quién lo hace cumplir.

Si venís de Java, tu instinto de encapsular está bien y no hay que apagarlo. Solo que ahora la **property de solo lectura con copia defensiva** es tu herramienta principal, no el modificador. Y la segunda herramienta —la que la mayoría no instala— es `mypy` corriendo en CI: sin él, `Final` es un comentario decorado.

Una objeción honesta

Llegado acá, el programador Java suele plantear la objeción obvia: si nada lo impide, ¿no es todo esto una ficción? ¿No es Python simplemente menos seguro?

La respuesta tiene dos partes. La primera es que el `private` de Java tampoco es absoluto: existe la reflexión, existe `setAccessible(true)`, y cualquier framework de serialización o de inyección de dependencias los usa todos los días para leer y escribir campos privados de tus clases. La barrera es alta, no infinita.

La segunda parte es más interesante. El encapsulamiento nunca fue, en el fondo, un mecanismo de seguridad: es un mecanismo de **comunicación**. Sirve para decirle al próximo programador —que muchas veces sos vos en seis meses— qué partes del objeto son contrato estable y qué partes son implementación que puede cambiar. Un atacante no es el destinatario del `private`; un colega lo es. Y para comunicar, un `_` que la comunidad entera entiende, que el linter marca y que el reviewer señala, comunica exactamente lo mismo que un `private`.

Lo que Python te quita no es el encapsulamiento: es la posibilidad de delegarlo. En Java podías no pensar en el tema y dejar que el compilador se encargara. En Python tenés que decidirlo, escribirlo y sostenerlo. Es más trabajo, y también es más honesto respecto de lo que el encapsulamiento siempre fue.

4 Lo que se traduce mal

Streams, sobrecarga y los reflejos que fallan

Este capítulo reúne las trampas que aparecen cuando traducís Java a Python «por reflejo»: casos donde el resultado funciona pero delata el origen, o donde directamente ni siquiera es Python. Tres de las cuatro fueron detectadas en material real de la cátedra —la sobrecarga es la excepción: es una trampa general del pasaje—, lo cual conviene decir sin dramatismo: son los errores que comete alguien que sabe lo que está haciendo y traduce en piloto automático. Por eso mismo valen como material didáctico.

4.1 · Streams → comprehensions

El pipeline de streams de Java y la comprehension de Python resuelven el mismo problema —transformar y filtrar colecciones sin escribir un for con acumulador— y llegaron a soluciones parecidas por caminos distintos. Python tuvo comprehensions desde el 2000; Java tuvo streams desde el 8, en 2014. La equivalencia es conceptual y casi siempre directa, pero la sintaxis no se parece en nada, y ahí es donde el reflejo falla.

Este es un caso real, sobreviviendo íntegro dentro de un manual de Python:

JAVA — INCORRECTO EN CONTEXTO PYTHON

```
Map<Comisario,Long> conteo = eventos.stream()
    .flatMap(e -> e.getComisarios().stream())
    .collect(groupingBy(c -> c, counting()));
```

PYTHON IDIOMÁTICO

```
from collections import Counter

conteo = Counter(c for e in eventos for c in e.comisarios)
maximo = max(conteo.values())
mas_frecuentes = [c for c, n in conteo.items() if n == maximo]
```

Vale la pena desarmar la conversión porque el patrón se repite. El `flatMap` —aplanar una colección de colecciones— se vuelve el **doblo for dentro de la comprehension**: `for e in eventos for c in e.comisarios`. Se lee en el mismo orden en que lo escribirías anidado, de afuera hacia adentro. Y el `collect(groupingBy(c -> c, counting()))` —agrupar por identidad y contar— es literalmente lo que hace `Counter`, así que desaparece: no se traduce, se reemplaza por la estructura de datos que existía para eso.

Notá que la versión Python no tiene un for explícito y sin embargo tampoco tiene un pipeline. Ese es el punto general: **casi todo pipeline de streams tiene una comprehension equivalente más corta**. Y cuando no la tiene —cuando la lógica es lo bastante compleja como para que la comprehension quede ilegible— el idioma correcto en Python es un for explícito, no una traducción forzada. Python no considera que el for sea una derrota; encadenar cinco operaciones funcionales para evitarlo, en cambio, sí es un java-ismo.

La conversión, pieza por pieza

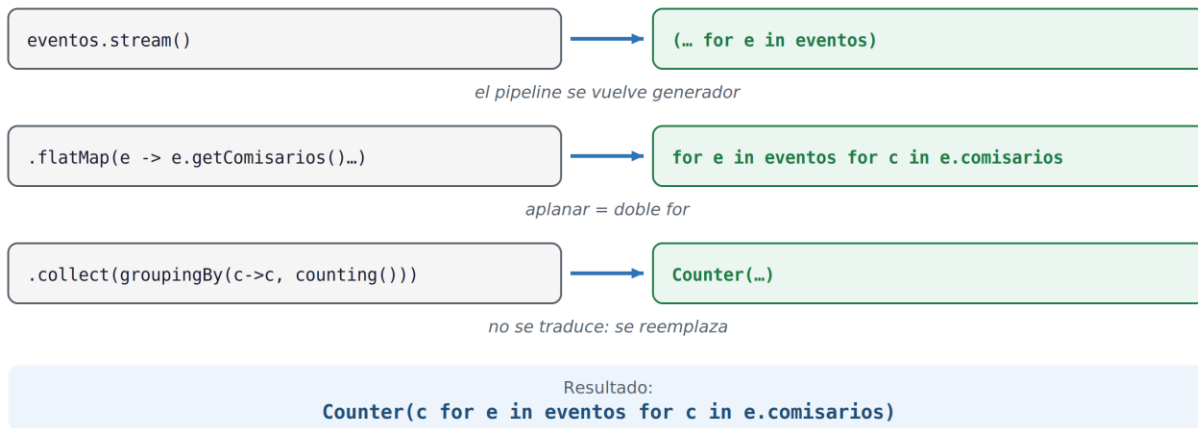


Figura 6 · La conversión pieza por pieza: el `flatMap` se vuelve `doble for`, y el `collect` no se traduce — se reemplaza por `Counter`.

Regla mental de conversión

Java Stream	Python
<code>.filter(x -> cond)</code>	<code>[x for x in xs if cond]</code>
<code>.map(x -> f(x))</code>	<code>[f(x) for x in xs]</code>
<code>.flatMap(...)</code>	doble for en la comprehension
<code>.collect(groupingBy(...))</code>	<code>Counter</code> / <code>defaultdict(list)</code>
<code>.mapToDouble(...).sum()</code>	<code>sum(f(x) for x in xs)</code>
<code>.anyMatch(...)</code> / <code>.allMatch(...)</code>	<code>any(...)</code> / <code>all(...)</code>
<code>.sorted(comparing(...))</code>	<code>sorted(xs, key=...)</code>
<code>.distinct()</code>	<code>set(xs)</code> o <code>dict.fromkeys(xs)</code>
<code>.reduce(...)</code>	<code>functools.reduce(...)</code> o <code>sum/min/max</code>
<code>.limit(n)</code>	<code>itertools.islice(xs, n)</code>
<code>.count()</code>	<code>len(xs)</code> o <code>sum(1 for _ in xs)</code>

4.2 · Sobrecarga de constructores

Java resuelve las variantes de construcción con múltiples firmas y deja que el compilador elija. Python no tiene sobrecarga: si escribís dos `__init__`, el segundo pisa al primero en silencio y el primero deja de existir. No hay error, no hay advertencia.

El idioma correcto es el **constructor alternativo** con `@classmethod`: un solo `__init__` que recibe lo esencial, y métodos de clase con nombre descriptivo para cada forma de construcción.

PYTHON

```
class Figura:
    def __init__(self, lados: list[Lado]) -> None:
        self._lados = lados

    @classmethod
    def vacia(cls) -> "Figura":
        return cls([])

    @classmethod
    def desde_medidas(cls, *medidas: float) -> "Figura":
        return cls([Lado(m) for m in medidas])
```

Fijate que esto es una mejora de legibilidad, no una concesión. En Java, `new Figura()` y `new Figura(3.0, 4.0, 5.0)` obligan al lector a mirar las firmas para saber cuál es cuál. En Python, `Figura.vacia()` y `Figura.desde_medidas(3, 4, 5)` se explican solos. El `cls` en lugar del nombre de la clase, además, hace que las subclases hereden los constructores alternativos y devuelvan instancias del tipo correcto — algo que en Java requiere trabajo extra.

4.3 · El punto y coma aplastado

Este es el reflejo más visible: escribir Python como si fuera Java al que se le borraron las llaves.

INCORRECTO — NO ES PYTHON

```
class Empleado: def __init__(self): self._rol_docente = None;
self._rol_investigador = None; self._rol_admin = None
```

PYTHON

```
class Empleado:
    def __init__(self) -> None:
        self._rol_docente: RolDocente | None = None
        self._rol_investigador: RolInvestigador | None = None
        self._rol_admin: RolAdmin | None = None

    def como_docente(self) -> RolDocente:
        if self._rol_docente is None:
            self._rol_docente = RolDocente()
        return self._rol_docente
```

La indentación **es** la sintaxis. El `;` existe en Python —es legal— pero está proscripto por convención, y PEP 8 lo dice explícitamente. Enseñar por contraejemplo su uso, en un manual que en el cuerpo usa type hints y properties impecables, es una incoherencia que el estudiante copia sin pensar, porque asume que si está en el material está bien.

El caso de arriba tiene además un segundo problema, más de fondo: la versión aplastada esconde que `como_docente()` es donde vive el patrón. La composición de roles del Ejercicio 6 —el *overlapping*— se entiende leyendo ese método, no la lista de asignaciones. Cuando aplastás el código, lo que se pierde no es solo el estilo: es la estructura que hacía visible el diseño.

4.4 · Los tipos

El residuo más común y más fácil de pasar por alto: `String` donde va `str`. Es menor y no rompe nada, pero delata el origen del texto y, en un manual, le enseña al estudiante un tipo que no existe.

Java	Python
<code>String</code>	<code>str</code>
<code>int / Integer</code>	<code>int</code>
<code>double / float</code>	<code>float</code>
<code>boolean / Boolean</code>	<code>bool</code>
<code>List<T></code>	<code>list[T]</code>
<code>Map<K,V></code>	<code>dict[K,V]</code>
<code>Set<T></code>	<code>set[T]</code>
<code>null</code>	<code>None</code>
<code>Object</code>	<code>object</code>
<code>Optional<T></code>	<code>T None</code>
<code>char</code>	<code>str</code> (de longitud 1)
<code>long</code>	<code>int</code> (sin límite)
<code>BigDecimal</code>	<code>decimal.Decimal</code>
<code>void</code>	<code>None</code> (como tipo de retorno)

Dos filas merecen atención. El `long` de Java tiene 64 bits y desborda; el `int` de Python tiene precisión arbitraria y no desborda nunca — un factorial de 100 se calcula sin trucos. Y el `char` no existe: un carácter es un `str` de longitud uno, lo que significa que iterar un string te da strings, no otra cosa.

5 Trampas que Java no te enseñó a ver

Cuatro errores que en Java simplemente no pueden pasar

Este es el capítulo más urgente en la práctica. Son errores que Java te *impedía* cometer — así que tu intuición de programador Java no los detecta. No los vas a ver venir. Y dos de ellos producen bugs que se manifiestan lejos, en el tiempo y en el archivo, del lugar donde los escribiste.

① Default mutable compartido

El clásico absoluto, el que todo programador Python pisa una vez y no olvida nunca. El valor por defecto de un parámetro se evalúa **una sola vez, cuando se define la función** — no en cada llamada. La lista vacía que escribiste como default es *una* lista, creada al importar el módulo, y todas las instancias la comparten.

```
PYTHON
# X TODAS las instancias comparten la MISMA lista
def __init__(self, lados: list[Lado] = []):
    self._lados = lados

# ✓ Correcto
def __init__(self, lados: list[Lado] | None = None):
    self._lados = lados if lados is not None else []
```

El síntoma es desconcertante: creás un Poligono nuevo, le agregás un lado, y aparece con los lados del polígono anterior. Como el defecto está en la *definición* y no en el uso, el bug se manifiesta a distancia y la sospecha nunca cae sobre la línea correcta.

```
CONSOLA — EL SÍNTOMA
>>> p1 = Poligono()
>>> p1.agregar_lado(Lado(3.0))
>>> p2 = Poligono()          # "nuevo", recién construido
>>> p2.nro_lados
1                             # ← ya nace con el lado de p1
>>> p1._lados is p2._lados
True                          # es literalmente la misma lista
```

La última línea es la que conviene mostrar en clase: `is` compara identidad, no contenido, y ese `True` dice que no hay dos listas parecidas sino una sola compartida. Es la evidencia directa de que el default se evaluó una única vez.

En Java esto no puede pasar porque el equivalente —`this(new ArrayList<>())`— ejecuta el `new` en cada llamada. Por eso tu intuición no tiene el reflejo. La regla mecánica es simple y no admite excepciones: **ningún default mutable, nunca**. Ni listas, ni diccionarios, ni sets, ni objetos. Siempre `None` y la construcción adentro.

El default se evalúa una sola vez

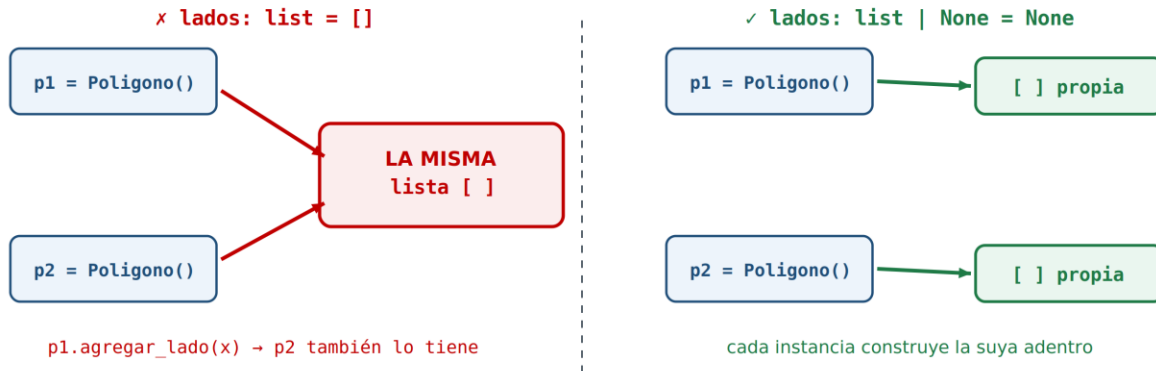


Figura 7 · El default se evalúa al definir la función: a la izquierda, ambas instancias comparten la misma lista; a la derecha, cada una construye la suya.

② Atributo de clase mutable — el static accidental

En Java, la diferencia entre un campo de instancia y un campo estático es una palabra clave visible que ninguna revisión pasa por alto. En Python la diferencia es la **indentación y el self** — dos cosas que el ojo entrenado en Java no lee como significativas.

PYTHON

```
class Poligono:
    lados: list[Lado] = []    # X es static: compartido por TODAS

    def __init__(self) -> None:
        self._lados: list[Lado] = []    # ✓ es de instancia
```

Lo insidioso es que la línea de arriba *parece* una declaración de campo al estilo Java —tipo, nombre, valor inicial— y es exactamente donde un programador Java esperaría declararlo. Pero en Python eso crea un atributo **de la clase**, no de la instancia: hay una sola lista para todos los polígonos del sistema.

La confusión se agrava porque con un `@dataclass` esa misma sintaxis *sí* declara campos de instancia — el decorador la reinterpreta. Y si intentás poner una lista como default en una `dataclass`, Python te tira un error explícito y te obliga a usar `field(default_factory=list)`. Es de los pocos lugares donde el lenguaje decidió protegerte activamente de esta trampa, precisamente porque es muy común.

③ `super().__init__()` olvidado

Java llama al constructor del padre automáticamente si no lo hacés vos: inserta un `super()` invisible al principio de tu constructor. **Python no.** Si no escribís `super().__init__()`, el constructor del padre simplemente no corre.

El resultado es un objeto a medio construir. En el ejemplo del Ejercicio 1, si `Poligono.__init__` se olvida del `super().__init__()`, todo lo que Figura iba a inicializar no existe — y el error aparece después, en el primer método que intente leer alguno de esos atributos, con un `AttributeError` que apunta a un lugar que no tiene nada que ver.

La regla práctica: **si tu clase hereda y define `__init__`**, la primera línea es `super().__init__(...)`. Sin excepciones y sin pensarlo. En jerarquías con herencia múltiple la llamada es además la que hace funcionar el MRO, así que saltarla rompe cosas más sutiles todavía.

④ Los type hints no validan nada

Esta es la trampa más peligrosa del capítulo, porque no produce un bug: produce **confianza infundada**.

`def agregar_lado(self, lado: Lado)` acepta un `str` sin chistar. Acepta `None`, acepta un `int`, acepta cualquier cosa. Los hints son documentación para el lector y metadatos para las herramientas; el intérprete los guarda en `__annotations__` y sigue de largo. No hay ninguna verificación en runtime, nunca.

El problema es que un código Python lleno de type hints *se parece* mucho a Java. Ves `def lados_esperados(self) -> int:` y una parte de tu cerebro registra «el compilador me cubre». No te cubre nadie. Si querés esa red, hay que instalarla:

SHELL

```
# En CI, no opcional:
mypy --strict src/
```

Sin mypy corriendo en CI, un manual o un proyecto lleno de type hints es **peor** que uno sin hints, porque genera la sensación de tener garantías que no existen. Con mypy, en cambio, los hints se vuelven algo bastante parecido a un compilador — y encima uno más expresivo que el de Java en varios aspectos, porque puede expresar `T | None`, tipos literales, protocolos estructurales y genéricos variantes sin borrado de tipos.

El resumen honesto: Python te da un sistema de tipos tan bueno como el de Java, pero *opcional*. Y opcional significa que la decisión de tenerlo es tuya, no del lenguaje.

6 Duck typing

Donde el diseño realmente cambia

Todo lo anterior es **traducción**. Esto es **rediseño**.

El nombre viene del dicho: si camina como un pato y hace cuac como un pato, es un pato. Aplicado al código: si el objeto tiene el método que necesito, me sirve, y no me importa de qué clase es ni de quién hereda.

El Ejercicio 7 de la serie usa lo que el manual llama «la superclase deducida»: el enunciado no menciona ninguna clase padre, pero al analizarlo se deduce que hay un concepto común y se lo modela. En Java, esa superclase es **obligatoria** — sin ella no hay tipo común, sin tipo común no hay lista que los contenga, y sin lista no hay polimorfismo. El compilador exige un ancestro compartido antes de dejarte tratar dos objetos igual.

En Python, el polimorfismo no necesita la herencia:

PYTHON

```
# Sin superclase común: si tiene calcular_area(), sirve
for f in [Triangulo(), Circulo(), Rectangulo()]:
    print(f.calcular_area())
```

Esas tres clases pueden no tener absolutamente nada en común. Pueden venir de tres librerías distintas, escritas por gente que no se conoce. Mientras las tres respondan a `calcular_area()`, el bucle funciona. No hay declaración, no hay implements, no hay registro.

El polimorfismo, con y sin superclase

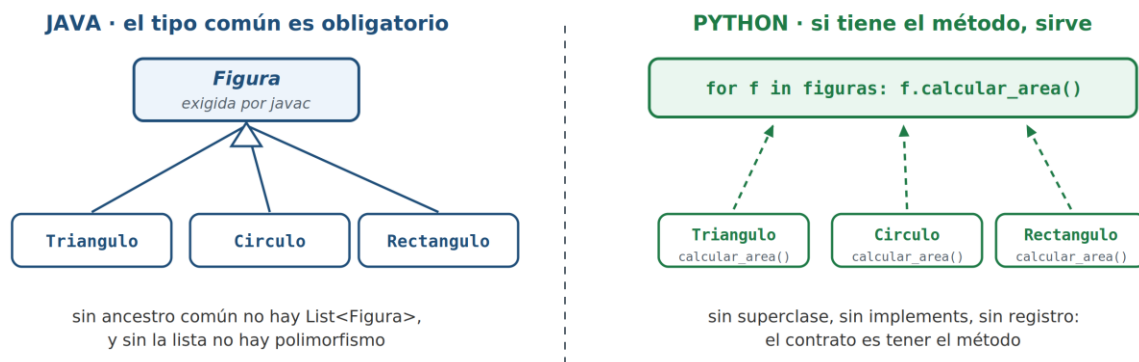


Figura 8 · A la izquierda, la superclase que javac exige para el tipo común. A la derecha, el bucle que solo pide que el método exista.

Pero esto NO invalida tu modelo UML

Y acá está la lección central del capítulo, la que hay que resistirse a simplificar.

El reflejo, al descubrir el duck typing, es concluir que la herencia sobra y que las jerarquías del manual eran andamiaje del compilador. Es una conclusión apresurada. La superclase deducida del Ejercicio 7 no está ahí *solo* para que

compile: está ahí porque **el dominio dice que esas cosas son un mismo concepto**. El UML modela el dominio, no las restricciones de javac. La superclase se justifica por semántica, no por sintaxis.

La diferencia es que en Java las dos razones —«el dominio lo dice» y «sin esto no compila»— producen exactamente el mismo código, y por lo tanto se confunden. Nunca tenés que elegir cuál era. En Python se separan: la razón sintáctica desaparece y solo queda la semántica. Y esa separación es un regalo pedagógico, porque **revela cuáles de tus jerarquías eran diseño y cuáles eran ceremonia**. Es el mismo experimento que hacen los dos manuales con los diagramas idénticos, pero aplicado a la herencia.

¿Cuándo la herencia sigue valiendo la pena en Python?

- ✓ Cuando comparte **implementación** — hay código real en el padre que las subclases reusan. Esta razón nunca fue sintáctica y no cambia.
- ✓ Cuando el **dominio afirma** el «es-un» — tu criterio del manual sigue intacto, palabra por palabra.
- ✓ Cuando querés que `@abstractmethod` te falle temprano — la ABC impide instanciar si falta una operación, y ese error al construir vale más que un `AttributeError` tres capas después.
- ✗ Cuando la única razón era «necesito un tipo común para el polimorfismo» → en Python eso es gratis, y la jerarquía es peso muerto que acopla sin dar nada.

El tercer punto de la lista —que la ABC falle temprano— se entiende mejor viendo los dos modos de fallo lado a lado. Con duck typing puro, el olvido de un método se descubre *al llamarlo*, en runtime, quizá en producción. Con la ABC, se descubre *al construir*, con un mensaje que nombra exactamente lo que falta:

PYTHON — DOS MODOS DE FALLO

```
# Duck typing puro: el error espera hasta la llamada
class Pentagono:          # olvidamos calcular_area()
    ...

>>> f = Pentagono()      # construye sin quejarse
>>> f.calcular_area()    # explota acá, quizá mucho después
AttributeError: 'Pentagono' object has no attribute 'calcular_area'

# Con ABC: el error no espera
class Pentagono(Poligono): # Poligono es ABC con @abstractmethod
    ...                  # olvidamos lados_esperados()

>>> f = Pentagono()      # explota al construir
TypeError: Can't instantiate abstract class Pentagono without an
implementation for abstract method 'lados_esperados'
```

Los dos mensajes son reales, copiados del intérprete. La diferencia entre ellos es distancia: el `AttributeError` aparece donde se usa el objeto, que puede estar a tres módulos del error; el `TypeError` aparece donde se construye, que casi siempre está al lado. Ese acortamiento de distancia es lo que comprás cuando elegís la ABC — y es una razón de diseño legítima, independiente del compilador.

Protocol: el contrato sin el acoplamiento

El duck typing puro tiene un costo evidente: no hay contrato. Nada documenta que el bucle espera un `calcular_area()`, y si le pasás un objeto que no lo tiene, te enterás en runtime.

Para eso está Protocol, que es tipado **estructural** verificable:

PYTHON

```
from typing import Protocol
```

```
class Dibujable(Protocol):          # nadie hereda de esto
    def dibujar(self) -> None: ...
```

```
def render(x: Dibujable) -> None: # mypy verifica que x tenga dibujar()
    x.dibujar()
```

La diferencia con una interfaz Java está en la dirección de la dependencia, y es más profunda de lo que parece. En Java, la clase **declara** que implementa la interfaz: `class Triangulo implements Dibujable`. El que cumple el contrato tiene que conocer el contrato. En Python con Protocol, la clase **no se entera de que el protocolo existe**. Cumple con tener el método; el protocolo lo verifica desde afuera.

Esto tiene una consecuencia práctica enorme: podés tipar código de terceros. Si una librería te da una clase que tiene `dibujar()` pero no hereda de nada tuyo, en Java no podés hacer nada —no podés modificar su código para agregarle el `implements`— y terminás escribiendo un adaptador. En Python definís el Protocol y esa clase ajena ya lo cumple, retroactivamente, sin tocarla.

Protocol es, entonces, duck typing **verificable**: el contrato sin el acoplamiento del `implements`. Java 8 agregó `default methods` en interfaces, que se acercan a las ABCs con implementación — pero seguís necesitando declarar `implements`. Esa declaración es justamente lo que Python no pide.

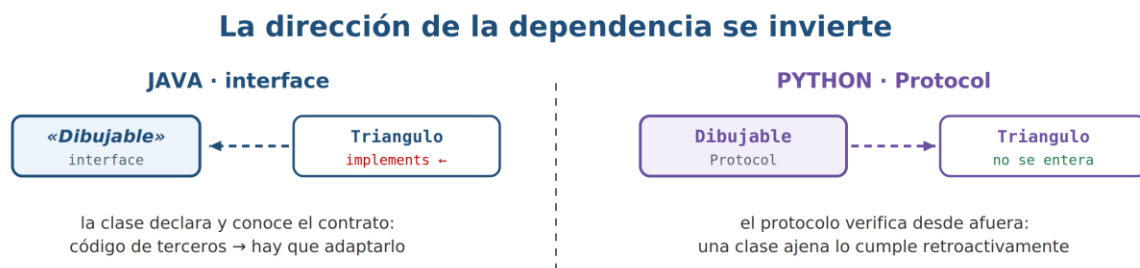


Figura 9 · La flecha se invierte: en Java la clase declara la interfaz; con Protocol, el contrato verifica desde afuera y la clase no se entera.

Entonces, ¿ABC o Protocol?

La regla que funciona: **ABC cuando sos dueño de la jerarquía y hay implementación para compartir; Protocol cuando definís un contrato que otros cumplen sin coordinarse con vos**. El Polígono del Ejercicio 1 es una ABC: tiene código real que las subclases reusan y el dominio afirma la jerarquía. Un Dibujable que atraviesa módulos y que quizá cumplan clases de terceros es un Protocol. Y las dos cosas pueden convivir en el mismo sistema sin conflicto.

7 Regalos que Java no tiene

dataclass, herencia múltiple, operadores

Hasta acá el manual se ocupó de lo que se pierde o de lo que hay que repensar. Este capítulo va en la otra dirección: qué te da Python que en Java no tenés, o que en Java requiere trabajo. No es una lista de curiosidades — son herramientas que cambian cómo se escribe el modelo.

dataclass

Campos, `__init__`, `__eq__` y `__repr__` en tres líneas. Todo el boilerplate que en Java te genera el IDE y que después tenés que mantener a mano cada vez que agregás un campo:

```
PYTHON
from dataclasses import dataclass

@dataclass(frozen=True)    # frozen = inmutable de verdad
class Lado:
    longitud: float
    color: str = "negro"

# Lado(3.0) == Lado(3.0) → True
# Sin escribir equals(), hashCode(), toString().
```

La diferencia con el código generado por el IDE es que el generado *es* código: vive en tu archivo, ocupa cuarenta líneas, y el día que agregás un campo tenés que acordarte de regenerarlo o el `equals` queda mal en silencio. La `dataclass` no genera texto: deriva el comportamiento de la declaración de campos, así que **no se puede desincronizar**.

El `frozen=True` merece un párrafo aparte porque no tiene equivalente Java. Hace que la instancia sea inmutable de verdad: asignar a un campo tira `FrozenInstanceError`. Java te da `final` campo por campo y aun así el objeto puede tener métodos que muten estructuras internas. El `frozen` es una propiedad de la clase entera, declarada en un lugar.

Herencia múltiple real

El Ejercicio 6 modela **overlapping**: un mismo empleado puede tener varios roles simultáneos. Java te obliga a resolverlo con interfaces o con composición, porque no permite heredar de dos clases. Python permite:

```
PYTHON
class Empleado(RolDocente, RolInvestigador, RolAdmin): ...
```

Funciona. El MRO —el algoritmo C3 que linealiza la jerarquía— resuelve el orden de resolución de métodos de forma determinista, y las tres clases aportan su implementación. Es la solución que un programador Java frustrado por años de no poder hacerlo va a querer usar el primer día.

CUIDADO — EL CRITERIO NO CAMBIÓ

Que Python *permita* herencia múltiple no la vuelve la mejor respuesta al overlapping. La **composición de roles opcionales que ya usás en el Ejercicio 6 sigue siendo el mejor modelo** — porque los roles se adquieren y se pierden en runtime, y la herencia es estática. Un empleado que deja de ser investigador tendría que cambiar de clase, lo cual es absurdo. Python te da la opción; el criterio de diseño no cambió.

Overlapping: dos formas de modelar los roles

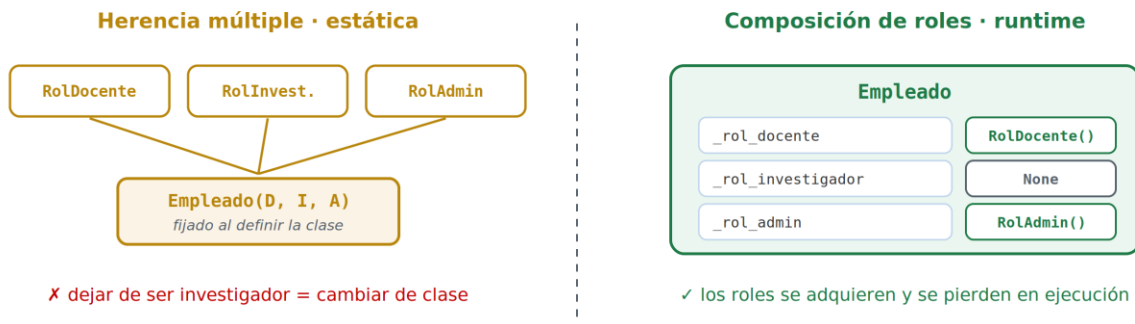


Figura 10 · El overlapping del Ejercicio 6: la herencia múltiple fija los roles al definir la clase; la composición los deja cambiar en ejecución.

Este caso es el mejor ejemplo del manual de que **la capacidad del lenguaje no es el criterio de diseño**. La pregunta nunca fue «¿puedo heredar de tres clases?». La pregunta era «¿los roles de un empleado son parte de su identidad permanente o son estados que cambian?». El enunciado responde que cambian. Con esa respuesta, la composición gana en Java y gana en Python, y que Python permita la alternativa es irrelevante.

Otros regalos

Recurso	Qué te da	Análogo Java
Sobrecarga de operadores	<code>__add__</code> , <code>__len__</code> , <code>__eq__</code> , <code>__lt__</code>	No existe
Context manager	<code>with</code> – <code>__enter__</code> / <code>__exit__</code>	<code>try-with-resources</code>
<code>@singledispatch</code>	Visitor sin boilerplate	Patrón Visitor completo
<code>@cached_property</code>	Derivado calculado una vez	Campo + lazy init a mano
Enum con comportamiento	Métodos y estado en el enum	Similar, más verboso
<code>__slots__</code>	Fija los atributos, ahorra memoria	Campos declarados (default)
Funciones de primera clase	Pasar métodos sin envolver	Lambdas / referencias a método

Dos de estos cambian cómo se ve el modelo. La **sobrecarga de operadores** permite que un `Medida` o un `Dinero` del dominio se sumen con `+`, que es como se escriben en el enunciado y en la cabeza del experto del dominio. Java te obliga a `a.sumar(b)`. Y `@cached_property` resuelve el atributo derivado costoso —el que en el UML lleva la barra

pero que no querés recalcular cada vez— en una línea, sin el campo auxiliar y el `if` (`cache == null`) que Java necesita.

El operador merece verse una vez completo, porque combina tres cosas del capítulo en diez líneas — `dataclass`, inmutabilidad y `dunder`:

PYTHON — EL DOMINIO SE ESCRIBE COMO SE DICE

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Medida:
    valor: float
    unidad: str

    def __add__(self, otra: "Medida") -> "Medida":
        if self.unidad != otra.unidad:
            raise ValueError("unidades incompatibles")
        return Medida(self.valor + otra.valor, self.unidad)

>>> Medida(3, "m") + Medida(4, "m")
Medida(valor=7, unidad='m')
>>> Medida(3, "m") + Medida(4, "kg")
ValueError: unidades incompatibles
```

Notá que el `__add__` devuelve una `Medida` nueva en lugar de modificar la existente — obligado por el `frozen`, y correcto para un objeto-valor: dos medidas se suman como se suman dos números, produciendo un tercero. El `__eq__` y el `__repr__` de la salida vinieron gratis con la `dataclass`. En Java, este mismo objeto-valor son unas cuarenta líneas.

8 Checklist de desintoxicación

Los 7 java-ismos a buscar en tu código Python

Usalo como checklist de code review — tuya y de tus estudiantes. Si venís de Java, estos siete reflejos son los que producen «Java escrito en Python»: código que funciona, pasa los tests, y le grita al lector de dónde viene.

No están en orden de gravedad sino de frecuencia. El primero es, de lejos, el más común.

1	¿Escribí <code>get_x()</code> / <code>set_x()</code> ?	→ Atributo público, o <code>@property</code> si hay lógica
2	¿Puse <code>__doble_guion</code> creyendo que es <code>private</code> ?	→ <code>_simple</code> es la convención
3	¿Heredé de una clase base solo para tener un tipo común?	→ Duck typing o Protocol
4	¿Definí una interfaz vacía para hacer implements?	→ Protocol
5	¿Escribí <code>__init__</code> que solo asigna campos?	→ <code>@dataclass</code>
6	¿Traduje un stream con <code>for</code> + <code>append</code> + acumulador?	→ Comprehension
7	¿Confío en que los type hints me protegen?	→ Corré mypy, o no te protegen

Cómo usar el checklist en clase

Los siete puntos tienen una estructura común que conviene explicitar cuando se enseña: cada uno es **un hábito que era correcto en Java**. Ninguno es ignorancia. El estudiante que escribe getters en Python no es que no sepa: es que sabe demasiado bien otra cosa. Esa es la diferencia entre corregir un error y desactivar un reflejo, y explica por qué el checklist no alcanza sin los capítulos que lo preceden. Marcar el punto 1 en un code review sin poder explicar el capítulo 2 produce obediencia, no comprensión — y el reflejo vuelve en el próximo archivo.

Cierre · Qué es UML y qué es sintaxis

El experimento controlado de los dos manuales

Los veintitrés diagramas idénticos entre el manual Java y el manual Python son el argumento entero de esta guía, y vale la pena decir por qué son un argumento y no una casualidad de producción.

Cuando dos manuales resuelven los mismos once enunciados en dos lenguajes distintos y llegan al mismo diagrama, tenés un **experimento controlado**. La variable independiente es el lenguaje; todo lo demás está fijo — el enunciado, el analista, el método de las tres lentes, los ocho patrones. Y el resultado es que el modelo no se movió un milímetro: clases, composición contra agregación, {XOR}, clase asociación, atributos derivados, generalización ortogonal, todo idéntico.

Lo que sí se movió fueron tres cosas, y ninguna es diseño:

- Los **modificadores**: `private/final` → convención, `property` y `mypy`. Cambió el mecanismo de cumplimiento, no la decisión de qué es interno.
- Las **ceremonias**: `getters`, `equals`, `hashCode` → `property` y `dataclass`. Desapareció el boilerplate, no el concepto de igualdad ni el de acceso controlado.
- La **obligatoriedad de la herencia**: requisito del compilador → decisión de dominio. La herencia no desapareció; dejó de ser gratuita y pasó a tener que justificarse.

El Ejercicio 7 es la prueba más limpia. La superclase deducida sobrevive al pasaje, y sobrevive *aunque Python no la necesite*. Si hubiera sido una concesión al compilador, en Python se habría evaporado. No se evaporó, porque nunca fue eso: era una afirmación sobre el dominio, y los dominios no cambian cuando cambiás de lenguaje. Lo que cae en Python es únicamente lo que en Java estaba ahí *para que compilara*.

Ahí está el valor pedagógico de tener las dos versiones. Un estudiante que solo vio el manual Java no tiene forma de distinguir qué parte de lo que aprendió era diseño y qué parte era Java: todo llega junto, en el mismo código, con la misma cara de necesario. Poner los dos manuales lado a lado, con el mismo diagrama arriba y dos códigos abajo, hace visible esa frontera. Lo que aparece en las dos versiones era modelo. Lo que aparece en una sola era lenguaje.

El experimento controlado

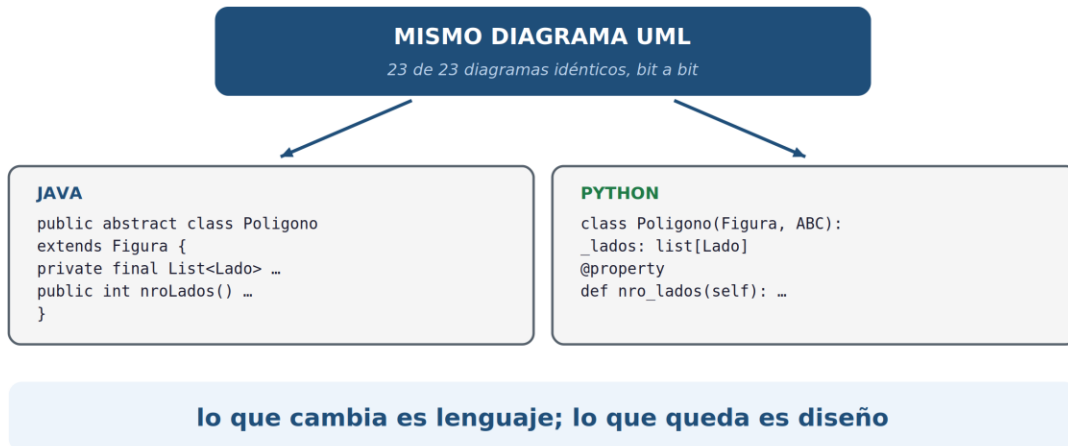


Figura 11 · El experimento: variable independiente el lenguaje, todo lo demás fijo. El diagrama no se mueve.

Poner los dos manuales lado a lado, con el mismo diagrama y dos códigos, es el experimento controlado:

lo que cambia es lenguaje; lo que queda es diseño.

Anexo A Tabla de conversión rápida

Para tener al lado del teclado

Referencia mecánica. A diferencia del capítulo 1, acá no hay criterio de diseño en juego: son correspondencias directas que conviene tener a mano hasta que salgan solas.

Convenciones de nombre

Elemento	Java	Python
Clase	Poligono	Poligono (igual)
Método	ladosEsperados()	lados_esperados()
Atributo	nroLados	nro_lados
Constante	MAX_LADOS	MAX_LADOS (igual)
Módulo / paquete	ar.utn.figuras	figuras.py
Booleano	esValida()	es_valida (property)

La regla es: **CamelCase para clases, snake_case para todo lo demás**. Es la convención de PEP 8 y no es negociable en un proyecto Python — un `ladosEsperados()` en Python se ve tan fuera de lugar como un `lados_esperados()` en Java.

Métodos especiales (dunder)

Java	Python	Se dispara con
<code>toString()</code>	<code>__str__</code> / <code>__repr__</code>	<code>str(x)</code> / <code>repr(x)</code>
<code>equals(o)</code>	<code>__eq__(self, o)</code>	<code>x == y</code>
<code>hashCode()</code>	<code>__hash__(self)</code>	<code>hash(x)</code> , uso en <code>set/dict</code>
<code>compareTo(o)</code>	<code>__lt__(self, o)</code>	<code>x < y</code> , <code>sorted()</code>
<code>size()</code> / <code>length()</code>	<code>__len__(self)</code>	<code>len(x)</code>
<code>iterator()</code>	<code>__iter__(self)</code>	<code>for i in x</code>
<code>contains(o)</code>	<code>__contains__(self, o)</code>	<code>o in x</code>
<code>get(i)</code>	<code>__getitem__(self, i)</code>	<code>x[i]</code>
(constructor)	<code>__init__(self)</code>	<code>Clase(...)</code>
<code>AutoCloseable</code>	<code>__enter__</code> / <code>__exit__</code>	<code>with x:</code>

La diferencia entre `__str__` y `__repr__` no tiene equivalente en Java y vale conocerla: `__str__` es para el usuario final, `__repr__` es para el programador — idealmente, el texto que reconstruiría el objeto. Si definís uno solo, definí `__repr__`: Python lo usa como fallback del otro, y es el que ves en el debugger y en la consola.

Del UML al código

Esta es la tabla que cierra el círculo del manual: cada elemento del diagrama, con sus dos implementaciones.

Elemento UML	Java	Python
Clase abstracta	<code>abstract class</code>	<code>class X(ABC)</code>
Operación abstracta	<code>abstract int m();</code>	<code>@abstractmethod</code>
/atributo derivado	<code>int getX() { ... }</code>	<code>@property def x</code>
Composición	List<Parte> en el todo	<code>self._partes: list[Parte]</code>
Agregación	List<Parte> en el todo	<code>self._partes: list[Parte]</code>
Generalización	<code>extends</code>	<code>class Sub(Super)</code>
Realización (interfaz)	<code>implements</code>	<code>Protocol (estructural)</code>
Clase asociación	clase con refs a A y B	clase con refs a A y B
Multiplicidad 0..1	<code>Optional<T> / null</code>	<code>T None</code>
Restricción {XOR}	validación + excepción	validación + <code>raise</code>
Rol de asociación	campo con nombre del rol	atributo con nombre del rol

Notá las filas de composición y agregación: **en los dos lenguajes se escriben igual**. La diferencia entre que la parte muera con el todo o le sobreviva no está en el código sino en quién construye y quién destruye — y eso es una decisión de diseño que ningún lenguaje expresa en la declaración del campo. Es exactamente el tipo de cosa que el diagrama dice y el código no, y la razón por la que el modelo no es redundante con la implementación.

Anexo B Autoevaluación y desafíos

Con soluciones

Como en el resto de la serie: intentá responder antes de mirar. El error propio enseña más que la respuesta dada.

Autoevaluación

1. En Java escribís getters «por las dudas». ¿Por qué en Python es un antipatrón?

Porque el getter Java existe como seguro contra el futuro: si el campo pasa a necesitar lógica, convertirlo en método cambia la sintaxis de acceso y rompe a todos los clientes. Es defensa propia legítima. Python elimina ese riesgo con `@property`: podés convertir un atributo en método sin que ningún cliente se entere, porque la sintaxis de acceso es idéntica para ambos. Desaparecido el riesgo, el getter preventivo no compra nada y pasa a ser ruido puro. Y el ruido tiene un costo concreto: si todos los accesos se ven iguales, el lector no distingue cuáles tienen lógica detrás. Cuando solo los que hacen algo son `properties`, el decorador informa.

2. ¿Qué diferencia hay entre `self._x` y `self.__x`? ¿Alguno es `private`?

Ninguno es `private`. `self._x` es pura convención: un cartel de «no entres» que los linters marcan, los IDEs esconden del autocompletado y el intérprete ignora por completo. `self.__x` activa `name mangling`: se renombra a `_Clase__x`, mecanismo pensado para evitar que una subclase pise sin querer un atributo interno del padre en jerarquías profundas. No es privacidad: `obj._Clase__x` funciona desde cualquier lado. Usar el doble guión creyendo que reproduce el `private` de Java tiene además un costo — rompe la herencia legítima y complica tests y debugging. La convención es guión simple; el doble se reserva para el problema puntual de colisiones.

3. Tu modelo UML tiene una superclase deducida (Ej. 7). En Python el polimorfismo no la necesita. ¿La borras?

No. La superclase deducida no estaba ahí para satisfacer al compilador: está porque el dominio afirma que esas cosas son un mismo concepto. El UML modela el dominio, no las restricciones de `javac`. Lo que sí desaparece en Python es la herencia cuya única justificación era «necesito un tipo común para que compile el polimorfismo». La observación interesante es que en Java las dos razones producen el mismo código y por eso se confunden; en Python se separan, y esa separación revela cuáles de tus jerarquías eran diseño y cuáles eran ceremonia. El criterio siempre fue semántico; Java simplemente no te obligaba a notarlo.

4. ¿Por qué `def __init__(self, lados=[])` es un error, y en Java el equivalente no lo es?

Porque el valor por defecto se evalúa una sola vez, al definir la función, no en cada llamada. Hay una única lista, creada al importar el módulo, y todas las instancias la comparten: creás un polígono nuevo, le agregás un lado, y aparece con los lados del anterior. En Java no puede pasar porque el equivalente — `this(new ArrayList<>())` — ejecuta el `new` en cada invocación, y por eso tu intuición no tiene el reflejo. La corrección es `lados: list[Lado] | None = None` y construir la lista adentro. La regla no admite excepciones: ningún default mutable, nunca — ni listas, ni diccionarios, ni sets, ni objetos.

5. Tenés type hints en todo el código. ¿Estás protegido como en Java?

No. Los hints son documentación y metadatos: el intérprete los guarda en `__annotations__` y sigue de largo. Una función anotada lado: Lado acepta un `str`, un `None` o cualquier cosa, sin chistar. Solo protegen si corrés mypy, idealmente en CI. Y acá está lo peligroso: un código lleno de hints se parece mucho a Java, y una parte de tu cerebro registra que el compilador te cubre. Sin mypy es peor que no tener hints, porque genera confianza infundada. Con mypy, en cambio, obtenés un sistema de tipos comparable al de Java y en varios aspectos más expresivo. El resumen honesto: Python te da tipado tan bueno como Java, pero opcional — y opcional significa que la decisión es tuya.

6. ¿Cuándo Protocol es mejor que heredar de una ABC?

Cuando querés el contrato sin el acoplamiento. La ABC exige que la clase declare la herencia: el que cumple tiene que conocer el contrato. Protocol usa tipado estructural, así que la clase que lo cumple nunca se entera de que existe. La consecuencia práctica es grande: podés tipar código de terceros. Si una librería te da una clase con `dibujar()` que no hereda de nada tuyo, en Java escribís un adaptador; en Python definís el Protocol y esa clase ya lo cumple, retroactivamente, sin tocarla. La ABC sigue siendo mejor cuando sos dueño de la jerarquía, hay implementación real para compartir, y querés que `@abstractmethod` falle al instanciar. Las dos conviven sin conflicto.

Desafíos · orientación de la solución

Desafío 1. Tomá la clase Poligono del Ejercicio 1 y reescribila con `@dataclass`. ¿Qué ganás y qué perdés?

Ganás `__init__`, `__eq__` y `__repr__` derivados de la declaración de campos —no generados como texto, así que no se pueden desincronizar cuando agregás un campo— y con `frozen=True` obtenés inmutabilidad real, más fuerte que cualquier cosa que Java te dé sin escribir código. Perdés control fino sobre la inicialización: si necesitás lógica en el constructor tenés que usar `__post_init__`, y `frozen=True` vuelve incómodo mutar la lista de lados, que es justamente lo que `agregar_lado()` hace. La conclusión de diseño es la interesante: `dataclass` brilla en los objetos-valor del modelo (Lado, Punto, Medida) —los que se comparan por contenido y no tienen identidad— y estorba en las entidades con ciclo de vida y comportamiento (Poligono). Ese es exactamente el criterio `value object` contra `entity`, y no lo inventó Python: lo venías aplicando en Java.

Desafío 2. Implementá el overlapping del Ejercicio 6 de las dos formas: herencia múltiple y composición de roles. Defendé cuál va al manual.

La herencia múltiple (`class Empleado(RolDocente, RolInvestigador)`) compila, funciona y el MRO resuelve el orden de forma determinista. Pero fija los roles en tiempo de definición de la clase: un empleado que deja de ser investigador tendría que cambiar de clase, lo cual es absurdo. La composición de roles opcionales (`self._rol_docente: RolDocente | None`, más un `como_docente()` que lo crea perezosamente) modela que los roles se adquieren y se pierden en runtime, que es lo que el enunciado dice. Va al manual la composición. La lección que trasciende el ejercicio: la pregunta nunca fue «¿puedo heredar de tres clases?» sino «¿los roles son identidad permanente o estado que cambia?». Con esa respuesta, la composición gana en Java y gana en Python — que Python permita la alternativa es irrelevante. La capacidad del lenguaje no es el criterio de diseño.

Desafío 3. Transferí: tomá un stream Java con `groupBy` y `counting` y convertilo a Python sin usar `for` explícito.

La conversión canónica es Counter con una comprehension generadora: Counter(c for e in eventos for c in e.comisarios). Desarmando: el flatMap —aplanar una colección de colecciones— se vuelve el doble for dentro de la comprehension, que se lee en el mismo orden en que lo escribirías anidado; y el collect(groupingBy(c->c, counting())) no se traduce sino que se reemplaza, porque Counter es exactamente la estructura de datos que existía para eso. Para agrupar sin contar, el análogo es defaultdict(list). El punto general: casi todo pipeline de streams tiene una comprehension equivalente más corta. Y cuando no la tiene —cuando la comprehension quedaría ilegible— el idioma correcto en Python es un for explícito. Python no considera que el for sea una derrota; encadenar cinco operaciones funcionales para evitarlo, en cambio, sí es un java-ismo.

Anexo C Glosario

Los términos del ecosistema Python, para quien viene de Java

El pasaje de lenguaje trae un vocabulario nuevo que los textos suelen dar por sabido. Cada entrada incluye, cuando existe, el ancla en el mundo Java.

ABC (Abstract Base Class)

Clase base abstracta del módulo `abc`. Una clase con ABC en su cadena de herencia y métodos `@abstractmethod` no se puede instanciar hasta que una subclase los implemente. Es el análogo directo de `abstract class`.

Comprehension

Expresión que construye una lista, set, dict o generador a partir de otra colección: `[f(x) for x in xs if cond]`. Cubre el territorio de los streams de Java (cap. 4).

dataclass

Decorador que deriva `__init__`, `__eq__` y `__repr__` de la declaración de campos. Con `frozen=True` produce inmutabilidad real. El objeto-valor de cuarenta líneas de Java, en tres (cap. 7).

Duck typing

«Si tiene el método, sirve»: la aptitud de un objeto se juzga por lo que sabe hacer, no por su clase declarada. Elimina la necesidad del tipo común que Java exige para el polimorfismo (cap. 6).

Dunder

Contracción de double underscore: los métodos `__asi__` que el lenguaje invoca por vos (`__init__`, `__eq__`, `__len__`). Son el equivalente de los métodos especiales que Java fija por nombre (`toString`, `equals`) — pero cubren también operadores y sintaxis (+, len, in, for).

Linter

Herramienta que analiza el código sin ejecutarlo y marca violaciones de convención y errores probables (ruff, pylint, flake8). Parte del «equipo» que en Python reemplaza al compilador como guardián (cap. 0).

MRO (Method Resolution Order)

El orden en que Python busca un método en una jerarquía con herencia múltiple, calculado por el algoritmo C3. Se consulta con `Clase.__mro__`. Es lo que vuelve determinista a `class Empleado(RolDocente, RolInvestigador)` (cap. 7).

mypy

Verificador estático de tipos: lee los type hints y reporta incompatibilidades sin ejecutar el programa. Corriendo en CI es lo más parecido al javac que Python ofrece (caps. 3 y 5). Opcional — y esa opcionalidad es la decisión de diseño.

Name mangling

El renombre automático de `__atributo` a `_Clase__atributo`. Existe para evitar colisiones de nombres entre padre e hijo, no para privacidad (cap. 3).

PEP 8

El documento oficial de convenciones de estilo de Python: `snake_case`, indentación de 4 espacios, sin punto y coma. En un ecosistema donde el rigor es cultural, la convención compartida es infraestructura (cap. 0).

Property

Método que se accede con sintaxis de atributo, vía `@property`. Implementa el atributo derivado (/x) del UML y elimina la necesidad del getter preventivo (cap. 2).

Protocol

Contrato de tipado estructural (módulo typing): una clase lo cumple por tener los métodos, sin declararlo ni saberlo. El «interface sin implements» (cap. 6).

Type hint

Anotación de tipo (`x: int, -> bool`). El intérprete no la verifica: es documentación para el lector y entrada para mypy (cap. 5, trampa ④).